

CrossGL: A Hub-and-Spoke Compiler Infrastructure for Universal Shader Translation Across Heterogeneous GPU Architectures

Anonymous Author(s)

Withheld for Double-Blind Review

Abstract. The modern GPU programming landscape is deeply fragmented across incompatible shading languages: GLSL for OpenGL, HLSL for DirectX, Metal Shading Language for Apple platforms, SPIR-V for Vulkan, and CUDA and HIP for general-purpose GPU computing. Developers targeting multiple platforms must maintain separate shader codebases, creating substantial engineering overhead and portability barriers. Existing translation tools such as SPIRV-Cross, Tint, and Naga address only subsets of this problem, each supporting a limited number of source and target languages and requiring pairwise converters that scale quadratically with the number of supported languages.

We present CrossGL, a compiler infrastructure employing a hub-and-spoke translation architecture centered on a purpose-built intermediate shader language. By routing all translations through a universal abstract syntax tree derived from the CrossGL language, the system reduces the complexity of supporting N languages from $O(N^2)$ pairwise converters to $O(N)$. The CrossGL language captures shader semantics across 16 GPU pipeline stages, including modern ray tracing and mesh shader stages, through a rich type system supporting generics, pattern matching, and platform-agnostic semantic annotations.

We demonstrate the approach through prototype backends for several diverse languages—including GLSL, HLSL, Metal, SPIR-V, CUDA, HIP, Rust, Mojo, and Slang—showing that adding a new target language requires only implementing a single code generator against the universal AST, rather than converters for every other language.

Keywords: shader translation, cross-platform GPU computing, intermediate representation, compiler infrastructure, heterogeneous architectures

1 Introduction

The graphics processing unit (GPU) ecosystem has become increasingly fragmented as hardware vendors and platform owners have developed competing shading and compute languages. OpenGL applications use GLSL [13], Microsoft’s DirectX ecosystem relies on HLSL [2], Apple’s platforms require Metal Shading Language [3], and Vulkan defines SPIR-V as its portable binary intermediate

representation [1]. Beyond graphics, general-purpose GPU (GPGPU) computing adds further fragmentation: NVIDIA’s CUDA [8] dominates the market while AMD’s HIP [9] provides an alternative, and emerging languages such as Mojo [10] and Slang [12] introduce yet more options. Developers who wish to target multiple platforms must either maintain separate codebases for each language or employ ad hoc translation scripts, leading to duplicated engineering effort, inconsistent behavior across platforms, and significant portability barriers.

Several tools attempt to address this fragmentation. SPIRV-Cross [4] decompiles SPIR-V binaries to GLSL, HLSL, and Metal, but operates only in one direction and cannot target CUDA, HIP, Rust, or Mojo. Google’s Tint [5] serves the WebGPU ecosystem by compiling WGSL but is not designed as a general-purpose translator. Mozilla’s Naga [6] supports translation among WGSL, GLSL, SPIR-V, and Metal within the `wgpu` project but covers a limited language set. Microsoft’s DirectX Shader Compiler (DXC) [7] compiles HLSL to DXIL and SPIR-V but remains HLSL-centric. Crucially, each of these tools implements pairwise converters: supporting N languages with direct translation requires up to $N(N - 1)$ converters, a quadratic scaling that makes comprehensive cross-platform support impractical.

We introduce CrossGL, a compiler infrastructure that addresses this challenge through a *hub-and-spoke* translation model. At the center of the architecture sits the CrossGL language, a purpose-built intermediate shader language with a rich type system, explicit pipeline stage semantics, and support for modern GPU features. Any source language is first translated to CrossGL through a backend-specific parser and reverse code generator; the resulting CrossGL representation is then translated to any target language through a corresponding forward code generator. This design requires only $2N$ converters for N languages— N reverse converters (source to CrossGL) and N forward converters (CrossGL to target)—reducing the quadratic scaling to linear.

The CrossGL language was designed as a semantic superset of shader concepts across all supported platforms. It provides explicit syntax for 16 GPU pipeline stages spanning traditional graphics (vertex, fragment, geometry, tessellation), modern mesh and task shaders, and all six DirectX Raytracing (DXR) stages. Its type system encompasses sized integer and floating-point primitives, vector and matrix types, texture and sampler abstractions, generics with trait constraints, and Rust-inspired pattern matching. A dual abstract syntax tree (AST) architecture separates backend-specific representations, which faithfully capture source language semantics, from a universal AST that serves as the normalized form for code generation.

This paper makes the following contributions:

- The CrossGL intermediate shader language, a human-readable IR designed to capture GPU shader semantics across 16 pipeline stages, generics, pattern matching, and platform-agnostic semantic annotations.
- A hub-and-spoke translation architecture that reduces the complexity of cross-platform shader translation from $O(N^2)$ pairwise converters to $O(N)$,

where each new language requires only two converters regardless of how many languages are already supported.

- A dual-AST design that cleanly separates backend-specific language representations from a universal normalized form, enabling faithful round-trip translation.
- An extensible plugin architecture that enables addition of new backends without modification to the core translation engine, as demonstrated through prototype backends spanning graphics shaders, GPU compute, and general-purpose languages.
- An open-source reference implementation demonstrating the feasibility of the approach across diverse target languages including GLSL, HLSL, Metal, SPIR-V, CUDA, HIP, Rust, Mojo, and Slang.

The remainder of this paper is organized as follows. Section 2 surveys background and related work. Section 3 describes the CrossGL language design. Section 4 presents the system architecture. Section 5 details the code generation strategies. Section 6 evaluates the system. Section 7 discusses limitations and future directions. Section 8 concludes.

2 Background and Related Work

2.1 GPU Programming Landscape

The proliferation of GPU hardware across desktop, mobile, and cloud platforms has produced a diverse ecosystem of shading and compute languages, each tailored to a specific graphics API or vendor architecture. Table 1 summarizes the principal languages and their characteristics.

Table 1. Overview of GPU shading and compute languages

Language	Platform	Domain	Stages
GLSL [13]	OpenGL/Vulkan	Graphics/Compute	10
HLSL [2]	DirectX	Graphics/Compute	14
Metal [3]	Apple	Graphics/Compute	8
SPIR-V [1]	Vulkan	Binary IR	14
CUDA [8]	NVIDIA	Compute	1
HIP [9]	AMD/NVIDIA	Compute	1
Slang [12]	Cross-platform	Graphics/Compute	14
Mojo [10]	Cross-platform	Compute/ML	1
Rust GPU [11]	Cross-platform	Graphics/Compute	Varies

GLSL [13] is the standard shading language for OpenGL and can also be used with Vulkan through compilation to SPIR-V. HLSL [2] powers the DirectX

ecosystem and has recently gained SPIR-V support through Microsoft’s DXC compiler [7]. Apple’s Metal Shading Language [3] is required for GPU programming on iOS and macOS. On the compute side, CUDA [8] remains the dominant language for NVIDIA hardware, while AMD’s HIP [9] provides source-level compatibility with CUDA for AMD GPUs. Emerging languages include Slang [12], which introduces extensible shader programming abstractions, and Mojo [10], which targets high-performance computing and machine learning workloads.

These languages share many fundamental concepts—vectors, matrices, textures, samplers, control flow—but differ in syntax, type naming conventions, pipeline stage models, and semantic annotation mechanisms [22]. This overlap motivates an intermediate representation that can capture their shared semantics while preserving platform-specific distinctions.

2.2 Existing Translation Tools

Several tools have been developed to bridge the gaps between GPU shading languages. Table 2 compares their capabilities.

Table 2. Comparison of existing shader translation tools

Tool	Sources	Targets	Bidir.	Extensible
SPIRV-Cross [4]	1	3	No	No
Tint [5]	1	4	No	No
Naga [6]	3	4	Partial	No
DXC [7]	1	2	No	No
Slang [12]	1	3	No	Yes
CrossGL	9+	9+	Yes	Yes

SPIRV-Cross [4], maintained by the Khronos Group, decompiles SPIR-V binary modules into GLSL, HLSL, and Metal source code. While widely used in game engines and graphics frameworks, it operates only in one direction (from SPIR-V) and cannot target compute-oriented languages such as CUDA, HIP, or Mojo.

Tint [5] is Google’s shader compiler for the WebGPU standard. It compiles WGSL (WebGPU Shading Language) to SPIR-V, Metal, HLSL, and GLSL. However, it accepts only WGSL as input and serves primarily the web platform rather than native GPU development.

Naga [6] is the shader translation module within Mozilla’s **wgpu** project. It supports WGSL, GLSL, and SPIR-V as inputs and can emit WGSL, SPIR-V, Metal, HLSL, and GLSL. While more versatile than single-source tools, its language coverage excludes CUDA, HIP, Rust, Mojo, and Slang.

DXC [7] compiles HLSL to DXIL (DirectX Intermediate Language) and SPIR-V. It is HLSL-centric and does not support other source languages or non-DirectX/Vulkan targets.

Slang [12] introduces language-level mechanisms for extensible shading, compiling to HLSL, GLSL, and SPIR-V. It represents a new source language rather than a universal translator.

A common limitation across these tools is the pairwise translation model: each source-target pair requires a dedicated converter. For N languages, full coverage demands $N(N - 1)$ converters. CrossGL overcomes this through a hub-and-spoke architecture that centralizes translation through a single intermediate language.

2.3 Intermediate Representations for Shaders

Intermediate representations (IRs) have long been central to compiler design [14]. In the GPU domain, SPIR-V [1] serves as the standard binary IR for Vulkan, encoding shader programs in a structured binary format. While SPIR-V preserves rich type information and control flow structure, it is not human-readable, is designed primarily for Vulkan consumption, and does not naturally support bidirectional translation.

At a lower level, LLVM IR [15] provides a universal compilation target used by many languages, including CUDA (via NVPTX) and AMD GPU targets (via AMDGPU). However, LLVM IR operates at the instruction level and does not preserve high-level shader semantics such as pipeline stages, vertex attributes, or texture sampling operations, making it unsuitable as a shader-to-shader translation IR.

MLIR [16] offers a more extensible approach through its dialect system, which allows domain-specific abstractions to coexist within a common framework. While promising, MLIR does not yet include a comprehensive shader dialect that captures the full breadth of GPU pipeline stages and graphics-specific operations.

The Cg language [17], developed by NVIDIA, was an early attempt at a cross-platform shading language that could compile to both OpenGL and DirectX targets. While pioneering, Cg predated modern GPU features such as compute shaders, ray tracing, and mesh shaders, and has been deprecated since 2012.

The Vulkan Programming Guide [21] provides comprehensive documentation of the SPIR-V consumption model, and the GLSL specification [23] defines the textual source language from which many CrossGL type and operator conventions derive.

CrossGL addresses the gap between these approaches by providing a human-readable, high-level IR that preserves shader-specific semantics—pipeline stages, vertex semantics, texture operations, wave intrinsics, and ray tracing constructs—while remaining amenable to bidirectional translation across both graphics and compute languages.

3 The CrossGL Language

CrossGL is a purpose-built intermediate shader language designed to serve as the universal hub in the translation architecture. This section describes its design goals, syntax and type system, and shader stage model.

3.1 Language Design Goals

The design of CrossGL was guided by five principles:

Universal expressiveness. The language must be capable of representing any construct expressible in any supported target language. This includes not only common shader operations (vector arithmetic, texture sampling, control flow) but also platform-specific features such as HLSL semantic annotations, Metal address space qualifiers, and CUDA thread indexing.

Semantic preservation. Pipeline stage information, vertex attribute semantics, compute workgroup dimensions, and other GPU-specific metadata must be first-class constructs in the language rather than annotations that could be lost during translation.

Human readability. Unlike binary IRs such as SPIR-V, CrossGL is designed to be readable and writable by developers. This facilitates debugging, manual inspection of translation results, and direct authoring of cross-platform shaders.

Extensibility. New GPU features—ray tracing stages, mesh shaders, wave operations—must be expressible without breaking backward compatibility with existing shaders.

Minimalism. The language includes only constructs that map to real GPU operations, avoiding unnecessary abstractions that would complicate translation fidelity.

3.2 Syntax and Type System

CrossGL uses a C-family syntax augmented with shader-specific constructs. Listing 1.1 shows a basic vertex-fragment shader.

```

1  shader SimpleShader {
2      struct VertexInput {
3          vec3 position;
4          vec2 texCoord;
5      }
6      struct VertexOutput {
7          vec2 uv;
8          vec4 position;
9      }
10     struct FragmentOutput {
11         vec4 color;
12     }
13     vertex {

```

```

14     VertexOutput main(VertexInput input) {
15         VertexOutput output;
16         output.uv = input.texCoord;
17         output.position = vec4(input.position, 1.0);
18         return output;
19     }
20 }
21 fragment {
22     FragmentOutput main(VertexOutput input) {
23         FragmentOutput output;
24         float r = input.uv.x;
25         float g = input.uv.y;
26         output.color = vec4(r, g, 0.5, 1.0);
27         return output;
28     }
29 }
30 }

```

Listing 1.1. A simple CrossGL vertex-fragment shader

The top-level construct is the **shader** block, which encapsulates all declarations and pipeline stage definitions for a single shader program. Within a shader block, developers define data structures using **struct** declarations and implement pipeline stages using named blocks (**vertex**, **fragment**, **compute**, etc.). Each stage contains a **main** entry point function and may include helper functions.

Type system. CrossGL provides a comprehensive type system spanning scalar, vector, matrix, and GPU-specific types. Table 3 summarizes the type categories.

Table 3. CrossGL type system categories

Category	Types
Signed integers	i8, i16, i32, i64, int
Unsigned integers	u8, u16, u32, u64, uint
Floating point	f16, f32, f64, half, float, double
Boolean	bool
Float vectors	vec2, vec3, vec4
Int/uint vectors	ivec2–ivec4, uvec2–uvec4
Bool vectors	bvec2, bvec3, bvec4
Matrices	mat2–mat4, mat2x3, mat4x2, etc.
Texture/sampler	sampler2D, samplerCube, sampler3D
Generics	generic<T>, generic<T: Numeric>

Beyond the standard shader type vocabulary, CrossGL supports Rust-inspired features including `let/let mut` variable declarations, `match` expressions with pattern guards and destructuring, `generic<T>` parameterized types with trait constraints, and `for-in` range iteration. These features, while uncommon in traditional shading languages, enable expressive cross-platform abstractions and facilitate translation from languages like Rust and Mojo that employ similar constructs.

Semantic annotations. CrossGL uses `@semantic` attributes to annotate shader input/output variables with platform-agnostic semantics. During code generation, these annotations are mapped to platform-specific mechanisms: GLSL `layout(location=N)` qualifiers, HLSL `SV_POSITION` semantics, or Metal `[[attribute(N)]]` annotations.

3.3 Shader Stage Model

CrossGL explicitly models 16 GPU pipeline stages, organized into four execution categories. Table 4 enumerates the supported stages.

Table 4. CrossGL pipeline stages by execution model

Execution Model	Stages
Traditional Graphics	<code>vertex</code> , <code>fragment</code> , <code>geometry</code> , <code>tessellation_control</code> , <code>tessellation_evaluation</code>
Compute	<code>compute</code>
Modern Graphics	<code>mesh</code> , <code>task</code> , <code>object</code>
Ray Tracing	<code>ray_generation</code> , <code>ray_intersection</code> , <code>ray_closest_hit</code> , <code>ray_miss</code> , <code>ray_any_hit</code> , <code>ray_callable</code>

Compute stages support `layout()` qualifiers for specifying workgroup dimensions and `shared` memory declarations for inter-thread communication. Ray tracing stages provide access to intrinsic operations including `TraceRay`, `ReportIntersection`, and `IgnoreHit`. Wave/subgroup operations (e.g., `WaveActiveSum`, `WaveBroadcast`) are represented as first-class intrinsic calls that map to HLSL wave intrinsics, Metal SIMD group functions, or GLSL subgroup operations during code generation.

4 System Architecture

CrossGL employs a hub-and-spoke architecture with the CrossGL language at the center and backend-specific modules at the periphery. This section describes the translation model, the dual-AST design, the compilation pipeline, and the plugin system.

4.1 Hub-and-Spoke Translation Model

The fundamental design principle of CrossGL is that all translations route through a single intermediate language. Rather than implementing direct converters between every pair of supported languages, the system decomposes each translation into at most two steps: conversion from a source language to CrossGL, and conversion from CrossGL to a target language.

This hub-and-spoke model supports three translation modes:

Forward translation converts CrossGL source code directly to a target language. The `.cgl` file is tokenized by the CrossGL lexer, parsed into a universal AST, and then emitted as target source code by a backend-specific code generator.

Reverse translation converts a native shader into CrossGL. The source file is tokenized and parsed by a backend-specific lexer and parser into a backend-specific AST, which is then converted to CrossGL source code by a reverse code generator.

Cross-translation combines the reverse and forward paths to translate between two non-CrossGL languages. For example, translating an HLSL shader to Metal proceeds as: $\text{HLSL} \rightarrow \text{HLSL lexer/parser} \rightarrow \text{HLSL AST} \rightarrow \text{reverse codegen} \rightarrow \text{CrossGL source} \rightarrow \text{CrossGL lexer/parser} \rightarrow \text{universal AST} \rightarrow \text{Metal codegen} \rightarrow \text{Metal source}$.

Complexity analysis. For N supported languages, the hub-and-spoke model requires N forward code generators (CrossGL to each target) and N reverse converters (each source to CrossGL), totaling $2N$ converters. A direct pairwise approach would require $N(N - 1)$ converters. With the current $N = 10$ languages, CrossGL uses 20 converters instead of 90, a $4.5\times$ reduction. Each additional language requires only 2 new converters rather than $2(N - 1)$.

4.2 Dual-AST Design

The system maintains two distinct AST layers, each optimized for a different purpose.

The universal AST serves as the canonical representation for forward code generation. It defines over 60 node types organized into six categories:

1. **Type system nodes** (10 types): `PrimitiveType`, `VectorType`, `MatrixType`, `ArrayType`, `PointerType`, `ReferenceType`, `FunctionType`, `GenericType`, `NamedType`, and the base `TypeNode`.
2. **Program structure nodes**: `ShaderNode` (root), `StageNode` (per pipeline stage with entry point and local functions), `ImportNode`, `PreprocessorNode`.
3. **Declaration nodes**: `StructNode`, `FunctionNode`, `VariableNode`, `ConstantNode`, `EnumNode`, `GenericParameterNode`, `AttributeNode`.
4. **Statement nodes**: `BlockNode`, `AssignmentNode`, `IfNode`, `ForNode`, `WhileNode`, `MatchNode`, `SwitchNode`, `ReturnNode`, among others.
5. **Expression nodes**: `BinaryOpNode`, `UnaryOpNode`, `FunctionCallNode`, `MemberAccessNode`, `ArrayAccessNode`, `SwizzleNode`, `CastNode`, `ConstructorNode`, `LambdaNode`.

6. **GPU-specific nodes:** `TextureOpNode`, `AtomicOpNode`, `SyncNode`, `WaveOpNode`, `RayTracingOpNode`, `MeshOpNode`, `BufferOpNode`.

Every AST node supports a visitor pattern via an `accept()` method that dispatches based on class name, and carries an `annotations` dictionary for backend-specific metadata.

The backend ASTs (a shared common AST plus per-backend extensions) provide simpler, native-language-oriented representations used by the reverse translation path. Each backend parser produces an AST that faithfully mirrors the source language’s structure—for example, the HLSL backend AST preserves HLSL-specific concepts such as constant buffer declarations and semantic annotations in their native form. The common base provides approximately 30 shared node types that all backends inherit.

The rationale for two AST layers is separation of concerns: backend parsers need to capture source language semantics without loss, while the forward code generators need a normalized, feature-rich representation that abstracts over platform differences. The universal AST serves as this normalized form.

4.3 Translation Pipeline

The forward translation pipeline consists of three stages: lexical analysis, parsing, and code generation.

Lexical analysis. The CrossGL lexer uses a combined regular expression pattern with named groups, compiled once at initialization. It recognizes over 220 token types spanning comments, preprocessor directives, keywords, type names, operators, and punctuation. A keyword post-lookup phase maps identifiers to their keyword token types. Token caching avoids redundant tuple allocation for frequently occurring tokens, and error reporting includes line and column numbers with surrounding context.

Parsing. The CrossGL parser is a hand-written recursive descent parser—a deliberate choice over parser generators to retain full control over error messages and recovery strategies. Expression parsing uses precedence climbing across 12 levels, from assignment (lowest) through ternary, logical, bitwise, comparison, shift, additive, multiplicative, and unary operators to postfix and primary expressions (highest). The parser employs speculative parsing with state save/restore for ambiguous constructs such as distinguishing function declarations from variable declarations. Dedicated recognition logic identifies GPU intrinsics during postfix expression parsing, including wave operations, ray tracing intrinsics, mesh shader operations, and ray query methods.

Code generation. Each target backend provides a code generator class that traverses the universal AST and emits target language source code. All code generators follow a common pattern: a top-level `generate()` method iterates over the shader’s structs, globals, constant buffers, functions, and pipeline stages, delegating to `generate_function()`, `generate_statement()`, and `generate_expression()` methods. Type, semantic, and operator mappings are maintained as per-backend dictionaries.

Two registries coordinate the pipeline. The `SourceRegistry` maps file extensions to source specifications (lexer, parser, and optional reverse code generator factories), determining how input files are parsed. The `BackendRegistry` maps backend names and aliases to code generator classes, determining how output is generated.

4.4 Plugin Architecture

CrossGL supports dynamic backend discovery through a plugin system (`translator/plugin_loader.py`). At initialization, the system scans subdirectories of `crosstl/backend/` for `backend_spec.py` or `source_spec.py` modules, dynamically importing and registering them. This enables new backends to be added as drop-in packages without modifying core translation logic.

Each backend is described by a `BackendSpec` dataclass containing: a canonical name, a list of aliases (e.g., “hlsl” and “dx” for DirectX), supported file extensions, and a factory function for the code generator class. Both registries use lazy loading—backend modules are imported only when a translation targeting them is requested, keeping startup time minimal regardless of the number of installed backends.

5 Code Generation

This section details the type and semantic mapping mechanisms, the per-backend generation strategies, and the specialized SPIR-V assembly generator.

5.1 Type and Semantic Mapping

Each code generator maintains three lookup dictionaries that map CrossGL constructs to their target language equivalents: a type mapping, a semantic mapping, and a function mapping.

Type mapping. Table 5 illustrates how representative CrossGL types map across five major backends. CrossGL’s type names align with GLSL conventions, so GLSL mappings are largely identity transformations. Other backends require systematic renaming; for example, HLSL and Metal use the `floatN` convention for vectors, while Rust uses `fN` scalar types and explicit vector libraries. The centralized `ASTUtils` class provides a unified `map_primitive_type()` method with per-backend mappings for all nine backends.

Semantic mapping. Shader input/output semantics are mapped from CrossGL’s unified names to platform-specific mechanisms. Table 6 shows representative mappings. GLSL uses built-in variables (`gl_Position`) and layout qualifiers. HLSL uses system-value semantics (`SV_POSITION`). Metal uses C++ attributes (`[[position]]`).

Function mapping. Built-in shader functions are mapped between naming conventions. For example, CrossGL’s `lerp` maps to GLSL’s `mix` and HLSL’s `lerp`; `frac` maps to GLSL’s `fract` and HLSL’s `frac`; `rsqrt` maps to GLSL’s `inversesqrt`; and `tex2D` maps to GLSL’s `texture`.

Table 5. Representative type mappings across backends

CrossGL	GLSL	HLSL	Metal	CUDA	Rust
vec2	vec2	float2	float2	float2	Vec2
vec4	vec4	float4	float4	float4	Vec4
mat4	mat4	float4x4	float4x4	float4x4	Mat4
ivec3	ivec3	int3	int3	int3	IVec3
sampler2D	sampler2D	Texture2D	texture2d	—	—
float	float	float	float	float	f32
int	int	int	int	int	i32
void	void	void	void	void	()

Table 6. Representative semantic mappings across backends

CrossGL	GLSL	HLSL	Metal
gl_Position	gl_Position	SV_POSITION	[[position]]
gl_FragColor	layout(0)	SV_TARGET	[[color(0)]]
gl_VertexID	gl_VertexID	SV_VertexID	[[vertex_id]]
gl_FragCoord	gl_FragCoord	SV_Position	[[position]]

5.2 Per-Backend Strategies

While all code generators follow the common traversal pattern described in Section 4.3, each backend addresses platform-specific challenges:

GLSL requires emitting a `#version` directive, translating shader stages to `void main()` entry points, converting struct-based I/O to `layout(location=N)` in/out variable declarations, and mapping CrossGL’s unified semantics to GLSL built-in variables.

HLSL preserves semantic annotations on struct members (e.g., `float4 position : SV_POSITION`), generates constant buffer declarations with register bindings, and handles the distinction between `Texture2D` resources and `SamplerState` objects.

Metal translates shader entry points to functions with Metal’s stage attributes (`vertex`, `fragment`, `kernel`), introduces address space qualifiers (`device`, `constant`, `threadgroup`), and restructures function parameters to use Metal’s argument buffer model with `[[buffer(N)]]` and `[[texture(N)]]` attributes.

CUDA and HIP emit kernel functions with `__global__` qualifiers, translate compute thread indexing from CrossGL’s `gl_GlobalInvocationID` to `threadIdx/blockIdx/blockDim` expressions, and map shared memory declarations to `__shared__` variables. HIP generation closely mirrors CUDA, reflecting HIP’s source-level compatibility design.

Rust maps CrossGL types to Rust primitives (`float` $\rightarrow$ `f32`, `int` $\rightarrow$ `i32`, `void` $\rightarrow$ `()`), translates structs to Rust `struct` definitions with `pub` fields, and converts control flow constructs to Rust syntax including expression-based returns.

Mojo translates types to Mojo’s naming conventions (`float` $\rightarrow$ `Float32`, `int` $\rightarrow$ `Int32`, `void` $\rightarrow$ `None`), uses Mojo’s `fn` function syntax, and generates appropriate Python-like indentation-based structure.

5.3 SPIR-V Assembly Generation

The SPIR-V code generator is the most complex backend, as it targets a structured binary instruction format rather than a textual programming language. The generator emits textual SPIR-V assembly (human-readable form) that can be assembled into binary SPIR-V using the `spirv-as` tool from the SPIR-V Tools suite.

Key implementation aspects include:

ID allocation. Every SPIR-V entity (types, variables, functions, labels, instructions) requires a unique numeric ID. The generator maintains a monotonically increasing ID counter with bound tracking to produce valid `OpMemoryModel` headers.

Type registry. SPIR-V requires explicit type declarations before use. The generator maintains registries for primitive types, vector types, matrix types, struct types, pointer types, function types, and array types, each indexed by a canonical key to ensure type deduplication.

Structured control flow. SPIR-V mandates structured control flow using `OpSelectionMerge` and `OpLoopMerge` instructions that declare merge and continue blocks before branch instructions. The generator tracks merge points to ensure valid structured control flow.

Extended instructions. Mathematical functions map to the GLSL.std.450 extended instruction set, requiring an `OpExtInstImport` declaration and `OpExtInst` calls with appropriate opcode numbers.

The system optionally integrates with `spirv-val` for validation and `spirv-dis/spirv-as` for formatting, falling back gracefully when these external tools are unavailable.

6 Evaluation

We evaluate the CrossGL approach along three dimensions: the architectural scalability of the hub-and-spoke model, the ease with which new backends can be added to the system, and a case study demonstrating translation of a complex real-world shader.

6.1 Architectural Scalability

The central claim of the hub-and-spoke architecture is that it reduces the engineering effort of multi-language support from quadratic to linear. Table 7 quantifies this advantage.

Table 7. Scaling properties of hub-and-spoke vs. pairwise translation

Languages (N)	Hub-and-spoke ($2N$)	Pairwise ($N(N - 1)$)
5	10 converters	20 converters
10	20 converters	90 converters
15	30 converters	210 converters
20	40 converters	380 converters

The advantage grows superlinearly: at 10 languages the hub-and-spoke model requires $4.5\times$ fewer converters; at 20 languages the reduction reaches $9.5\times$. More importantly, the marginal cost of adding the $(N + 1)$ -th language is constant (2 converters) under the hub-and-spoke model, whereas pairwise translation requires $2N$ new converters. This property makes the approach increasingly attractive as the GPU language ecosystem continues to expand.

The architecture also provides a structural guarantee: any two languages in the system can interoperate through CrossGL as a pivot, even if neither language’s implementor was aware of the other. This emergent interoperability—a direct consequence of the shared IR—is impossible to achieve incrementally with pairwise converters.

6.2 Ease of Adding New Backends

A key design goal of CrossGL is that adding support for a new target language should be straightforward and self-contained. To evaluate this, we examine the structure required for a new backend and illustrate through the backends implemented as proof-of-concept demonstrations.

What a new backend requires. To add a new target language, a developer implements a single code generator class that traverses the universal AST and emits target language source. The code generator follows a standard template:

1. Define a **type mapping** dictionary (CrossGL types $\rightarrow$ target types).
2. Define a **semantic mapping** dictionary (CrossGL semantics $\rightarrow$ target annotations).
3. Implement `generate()`, `generate_function()`, `generate_statement()`, and `generate_expression()` methods that walk the AST and emit text.
4. Register the backend via a `BackendSpec` descriptor with name, aliases, and file extension.

No changes to the core lexer, parser, or universal AST are required. The plugin architecture discovers new backends automatically through directory scanning.

Demonstrated backends. We have implemented prototype backends for nine diverse languages to validate that the architecture accommodates a wide range of target paradigms:

- **Graphics shaders** (GLSL, HLSL, Metal, Slang): These backends demonstrate translation of full graphics pipeline semantics including vertex/fragment stages, texture sampling, and semantic annotations. Each target uses fundamentally different mechanisms for the same concepts (GLSL’s `layout` qualifiers vs. HLSL’s `SV_` semantics vs. Metal’s C++ attributes).
- **GPU compute** (CUDA, HIP, SPIR-V): These backends demonstrate translation of compute kernels, shared memory, thread synchronization, and work-group indexing. SPIR-V is particularly noteworthy as it targets a structured assembly format rather than a textual language.
- **General-purpose languages** (Rust, Mojo): These backends demonstrate that the architecture extends beyond traditional GPU languages, translating CrossGL constructs to general-purpose language idioms with fundamentally different type systems and syntax conventions.

These backends are not all equally complete—some cover only core shader features while others handle advanced pipeline stages—but they collectively demonstrate that the hub-and-spoke architecture and the CrossGL IR are sufficiently general to accommodate languages across the graphics, compute, and general-purpose spectrum. The key observation is that each backend was implemented independently, against the same universal AST, without requiring changes to any other backend or to the core infrastructure.

Bidirectional translation. For languages where reverse translation is implemented (source $\rightarrow$ CrossGL), a backend consists of an additional lexer, parser, backend-specific AST, and reverse code generator. This enables two-hop cross-translation between any pair of supported languages. The reverse path follows the same modular structure: each backend’s reverse translator is self-contained and does not depend on other backends.

6.3 Case Study: Cross-Platform PBR Shader

To demonstrate CrossGL on a realistic workload, we authored a Universal Physically Based Rendering (PBR) shader in CrossGL (488 lines). This shader implements a complete Cook-Torrance BRDF [18] following the methodology of Pharr et al. [19], including:

- GGX/Trowbridge-Reitz normal distribution function
- Smith’s geometry function with Schlick-GGX approximation
- Schlick Fresnel approximation
- Cascaded shadow mapping with four cascade levels
- Image-based lighting with prefiltered environment maps
- Parallax occlusion mapping and Reinhard tone mapping

The shader defines 10 data structures and multiple helper functions, exercising a substantial portion of CrossGL’s type system and control flow constructs. We translated this shader to the graphics-oriented backends (GLSL, HLSL, Metal) to verify that complex, real-world shader logic translates correctly through the hub-and-spoke pipeline.

The generated outputs use idiomatic constructs for each target platform: GLSL output includes appropriate `#version` directives, `layout` qualifiers, and `uniform` blocks; HLSL output uses `SV_` semantic annotations and `cbuffer` declarations; Metal output employs argument buffers with address space qualifiers and stage-specific attributes. The mathematical structure of the BRDF computations—including the GGX distribution function, Fresnel equations, and tone mapping operators—is preserved faithfully across all targets.

This case study demonstrates that the CrossGL IR is expressive enough to capture the semantics of a production-quality shader, and that the translation pipeline produces correct, readable output across platforms with fundamentally different shader models.

6.4 Engineering Effort Analysis

Table 8 characterizes the relative engineering effort for each component of the system.

Table 8. Engineering effort by component category

Component	Effort characterization
CrossGL language (AST + Lexer + Parser)	One-time investment; shared by all backends
Forward code generator (per backend)	Primary per-backend effort; follows template
Reverse translator (per backend)	Optional; enables bidirectional translation
Plugin registration	Minimal; single descriptor file
Core infrastructure changes	None required for new backends

The one-time investment in the CrossGL language definition, lexer, and parser is amortized across all backends. Each new backend’s primary effort is implementing the forward code generator, which consists largely of defining type-/semantic mapping dictionaries and implementing the AST-to-text traversal. The standardized code generator interface means that existing backends serve as templates for new ones, further reducing the marginal effort of supporting additional languages.

7 Discussion

7.1 Limitations

While the hub-and-spoke approach offers clear architectural advantages, several limitations should be acknowledged.

Translation fidelity vs. engineering cost. The hub-and-spoke model trades potential translation fidelity for engineering scalability. Cross-translation between two non-CrossGL languages requires a two-hop path (source $\rightarrow$ CrossGL $\rightarrow$ target), and the intermediate representation may not preserve backend-specific optimizations or idioms. A dedicated pairwise converter between two specific languages could, in principle, produce more idiomatic output. However, we argue that for the vast majority of use cases, the linear scaling advantage outweighs this cost, particularly as the number of supported languages grows.

IR expressiveness bounds. The CrossGL language, despite being designed as a superset of shader concepts, necessarily makes design choices that favor some languages over others. GLSL-derived naming conventions (e.g., `vec3`, `mat4`) are natural for graphics shader backends but less so for general-purpose languages like Rust or Mojo. Certain language-specific features—Rust’s ownership semantics, CUDA’s cooperative group primitives, Metal’s address space model—do not map cleanly to the IR and require approximation or loss during translation.

No optimization passes. The current architecture performs direct AST-to-text translation without intermediate optimization. Dead code elimination, constant folding, and other standard compiler optimizations are not applied. In practice, the target platform’s own compiler (e.g., the GPU driver’s shader compiler) performs these optimizations, but the generated source may be less readable than hand-written code.

Correctness scope. Our evaluation focuses on structural and syntactic correctness of generated code rather than runtime equivalence. Proving that translated shaders produce identical pixel output or compute results across all backends would require a formal semantics for CrossGL and each target language, which remains future work.

7.2 Future Work

The hub-and-spoke architecture opens several avenues for future research and development, building on established principles of structured software design for parallel and heterogeneous systems [20].

Optimization passes over the universal AST. The visitor pattern already present in the AST design facilitates the introduction of optimization passes—dead code elimination, constant propagation, common subexpression elimination—that would benefit all backends simultaneously. This is a key advantage of the centralized IR approach: optimizations implemented once are shared across all translation paths.

Formal translation semantics. Developing formal semantics for CrossGL and proving translation equivalence between source and target programs would provide stronger correctness guarantees. The hub-and-spoke model simplifies this challenge: proving correctness for $2N$ individual converters is more tractable than $N(N - 1)$ pairwise proofs.

Extending to heterogeneous accelerators. The CrossGL IR could be extended with constructs for tensor operations, automatic differentiation, and

ML-specific data types, positioning it as a universal translation layer for heterogeneous AI accelerators (GPUs, NPUs, TPUs). The modular backend architecture makes such extensions feasible without disrupting existing backends.

Binary output formats. Direct emission of binary SPIR-V and DXIL would eliminate dependencies on external assembler tools and enable tighter integration with graphics runtimes.

IDE and developer tooling. A Language Server Protocol (LSP) implementation for CrossGL would enable syntax highlighting, autocompletion, and error diagnostics, improving the experience for developers who author cross-platform shaders directly in CrossGL.

Community-driven backend expansion. The plugin architecture is designed to enable community-contributed backends. As new GPU languages emerge (e.g., WGSL for WebGPU), the constant marginal cost of integration makes it practical for the community to extend the system without centralized coordination.

8 Conclusion

The fragmentation of GPU shading and compute languages imposes substantial engineering burden on developers targeting multiple platforms. This paper presented CrossGL, a compiler infrastructure that addresses this challenge through a hub-and-spoke translation architecture centered on a purpose-built intermediate shader language.

The CrossGL language provides a human-readable, semantically rich IR that captures shader concepts across 16 GPU pipeline stages, including modern ray tracing and mesh shader stages, through a type system supporting generics, pattern matching, and platform-agnostic semantic annotations. The hub-and-spoke architecture reduces the number of required translators from $O(N^2)$ to $O(N)$, with each new language requiring only two converters regardless of how many languages are already supported. We demonstrated the feasibility of this approach through prototype backends spanning graphics shaders (GLSL, HLSL, Metal, Slang), GPU compute (CUDA, HIP, SPIR-V), and general-purpose languages (Rust, Mojo), showing that each backend can be implemented independently against the universal AST without modifying the core infrastructure.

The extensible plugin architecture ensures that the system can evolve alongside the GPU ecosystem: new backends are added as self-contained modules, discovered automatically, and immediately interoperable with all existing backends through the CrossGL pivot. As heterogeneous computing architectures proliferate—spanning GPUs, NPUs, TPUs, and specialized AI accelerators—the need for universal translation infrastructure will only grow. The hub-and-spoke model provides a principled, scalable foundation for bridging these diverse platforms through a single, shared intermediate representation.

Acknowledgments. Details withheld for double-blind review.

References

1. Khronos Group: SPIR-V Specification, Version 1.6, Revision 2. Khronos Group (2022). <https://registry.khronos.org/SPIR-V/>
2. Microsoft Corporation: HLSL Programming Guide. Microsoft Learn (2023). <https://learn.microsoft.com/en-us/windows/win32/direct3dhls/>
3. Apple Inc.: Metal Shading Language Specification, Version 3.1. Apple Inc. (2023). <https://developer.apple.com/metal/Metal-Shading-Language-Specification.pdf>
4. Khronos Group: SPIRV-Cross – A Practical Tool for SPIR-V Reflection and Disassembly. GitHub Repository (2023). <https://github.com/KhronosGroup/SPIRV-Cross>
5. Google: Tint – A Compiler for the WebGPU Shading Language (WGSL). Part of the Dawn project (2023). <https://dawn.googlesource.com/tint>
6. Mozilla: Naga – Universal Shader Translation in Rust. Part of the wgpu project (2023). <https://github.com/gfx-rs/wgpu/tree/trunk/naga>
7. Microsoft Corporation: DirectX Shader Compiler (DXC). GitHub Repository (2023). <https://github.com/microsoft/DirectXShaderCompiler>
8. Nickolls, J., Buck, I., Garland, M., Skadron, K.: Scalable Parallel Programming with CUDA. *ACM Queue* **6**(2), 40–53 (2008). doi:10.1145/1365490.1365500
9. Advanced Micro Devices, Inc.: HIP Programming Guide. AMD ROCm Documentation (2023). <https://rocm.docs.amd.com/projects/HIP/>
10. Modular Inc.: The Mojo Programming Language. Modular (2023). <https://www.modular.com/mojo>
11. Embark Studios: rust-gpu – Making Rust a First-Class Language and Ecosystem for GPU Shaders. GitHub Repository (2023). <https://github.com/EmbarkStudios/rust-gpu>
12. He, Y., Fatahalian, K., Foley, T.: Slang: Language Mechanisms for Extensible Real-Time Shading Systems. *ACM Trans. Graph.* **37**(4), Article 141 (2018). doi:10.1145/3197517.3201362
13. Kessenich, J., Sellers, G., Shreiner, D.: OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.5 with SPIR-V, 9th edn. Addison-Wesley, Boston (2016)
14. Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D.: Compilers: Principles, Techniques, and Tools, 2nd edn. Pearson/Addison-Wesley, Boston (2006)
15. Lattner, C., Adve, V.: LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In: *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pp. 75–86. IEEE Computer Society (2004). doi:10.1109/CGO.2004.1281665
16. Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., Riddle, R., Shpeisman, T., Vasilache, N., Zinenko, O.: MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In: *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pp. 2–14. IEEE (2021). doi:10.1109/CGO51591.2021.9370308
17. Mark, W.R., Glanville, R.S., Akeley, K., Kilgard, M.J.: Cg: A System for Programming Graphics Hardware in a C-like Language. *ACM Trans. Graph.* **22**(3), 896–907 (2003). doi:10.1145/882262.882362
18. Cook, R.L., Torrance, K.E.: A Reflectance Model for Computer Graphics. *ACM Trans. Graph.* **1**(1), 7–24 (1982). doi:10.1145/357290.357293
19. Pharr, M., Jakob, W., Humphreys, G.: *Physically Based Rendering: From Theory to Implementation*, 4th edn. MIT Press, Cambridge (2023)

20. McCool, M., Reinders, J., Robison, A.: Structured Parallel Programming: Patterns for Efficient Computation. Morgan Kaufmann, San Francisco (2012)
21. Sellers, G., Kessenich, J.: Vulkan Programming Guide: The Official Guide to Learning Vulkan. Addison-Wesley, Boston (2017)
22. Owens, J.D., Houston, M., Luebke, D., Green, S., Stone, J.E., Phillips, J.C.: GPU Computing. *Proceedings of the IEEE* **96**(5), 879–899 (2008). doi:10.1109/JPROC.2008.917757
23. Kessenich, J., Baldwin, D., Rost, R.: The OpenGL Shading Language, Version 4.60. Khronos Group (2018). <https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf>